



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Title of Thesis

Bachelor Thesis

Lars Hulsbergen

Thursday 24th September, 2026

Advisors: Yakup Koray Budanaz, Lorenzo Paleari

Department of Computer Science, ETH Zürich

Acknowledgments

Abstract

Contents

Contents	iii
1 Introduction	1
1.1 Motivation	1
1.2 Research Questions	1
1.3 Contributions	1
1.4 Thesis Outline	1
2 Background	2
2.1 HPCAgent-Bench	2
2.2 The Berkeley Dwarfs	2
2.3 Compiler Auto-Vectorization and Loop-Level Reasoning	2
2.4 Floating-Point Semantics and Compiler Flags	2
2.5 Code Representations	2
2.6 Iterative Solvers and Preconditioning	2
2.7 Agent Skills	2
3 Measurement Methodology	3
3.1 Platform and Toolchain	3
3.2 Protocol	3
3.3 Statistics and Noise	3
3.4 Threats to Validity	3
4 Language Comparison on LLR-40	4
4.1 Kernels and Representations	4
4.2 Coverage and Correctness	4
4.3 Performance Across Representations	4
4.4 Per-Class Findings	4
4.5 Outliers	4
5 Extracted Application Kernels	5

5.1	Selection by Berkeley Dwarf	5
5.2	Porting Methodology	5
5.3	Kernel Descriptions	5
5.4	Manifest Design Decisions	5
5.5	Validation Against Upstream	5
6	Language Comparison on the Extracted Kernels	6
6.1	Setup	6
6.2	How These Kernels Differ from LLR-40	6
6.3	Coverage and Correctness	6
6.4	Performance	6
6.5	Comparison with LLR-40	6
7	The Solver Subtrack and the Preconditioning Skill	7
7.1	Test Kernels	7
7.2	Grading Versus Convergence	7
7.3	The Freedom Verdict	7
7.4	Making a Frozen Preconditioner Fast	7
7.5	Skill Design	7
7.6	Evaluation of the Skill	7
8	Extending the CloudSC Loop-Nest Analysis	8
8.1	Prior Results	8
8.2	What Was Left Open	8
8.3	Extended Setup	8
8.4	Results	8
8.5	Implications for DaCe Code Generation	8
9	Discussion	9
9.1	Representation, Compiler or Algorithm?	9
9.2	Vectorizing Is Not the Same as Being Fast	9
9.3	What Elementwise Grading Permits	9
9.4	Limitations	9
10	Conclusion	10
10.1	Summary	10
10.2	Future Work	10
	Bibliography	11
	LLR-40 Full Results	12
	Extracted Kernel Catalogue	13
	Solver-Preconditioners Skill	14

Reproducibility	15
------------------------	-----------

Chapter 1

Introduction

- 1.1 Motivation**
- 1.2 Research Questions**
- 1.3 Contributions**
- 1.4 Thesis Outline**

Background

- 2.1 HPCAgent-Bench**
- 2.2 The Berkeley Dwarfs**
- 2.3 Compiler Auto-Vectorization and Loop-Level Reasoning**
- 2.4 Floating-Point Semantics and Compiler Flags**
- 2.5 Code Representations**
- 2.6 Iterative Solvers and Preconditioning**
- 2.7 Agent Skills**

Chapter 3

Measurement Methodology

- 3.1 Platform and Toolchain**
- 3.2 Protocol**
- 3.3 Statistics and Noise**
- 3.4 Threats to Validity**

Chapter 4

Language Comparison on LLR-40

- 4.1 Kernels and Representations
- 4.2 Coverage and Correctness
- 4.3 Performance Across Representations
- 4.4 Per-Class Findings
- 4.5 Outliers

Extracted Application Kernels

- 5.1 Selection by Berkeley Dwarf**
- 5.2 Porting Methodology**
- 5.3 Kernel Descriptions**
- 5.4 Manifest Design Decisions**
- 5.5 Validation Against Upstream**

Language Comparison on the Extracted Kernels

6.1 Setup

6.2 How These Kernels Differ from LLR-40

6.3 Coverage and Correctness

6.4 Performance

6.5 Comparison with LLR-40

The Solver Subtrack and the Preconditioning Skill

- 7.1 Test Kernels**
- 7.2 Grading Versus Convergence**
- 7.3 The Freedom Verdict**
- 7.4 Making a Frozen Preconditioner Fast**
- 7.5 Skill Design**
- 7.6 Evaluation of the Skill**

Extending the CloudSC Loop-Nest Analysis

8.1 Prior Results

8.2 What Was Left Open

8.3 Extended Setup

8.4 Results

8.5 Implications for DaCe Code Generation

Discussion

- 9.1 Representation, Compiler or Algorithm?**
- 9.2 Vectorizing Is Not the Same as Being Fast**
- 9.3 What Elementwise Grading Permits**
- 9.4 Limitations**

Chapter 10

Conclusion

10.1 Summary

10.2 Future Work

Bibliography

LLR-40 Full Results

Extracted Kernel Catalogue

Solver-Preconditioners Skill

Reproducibility



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

- ☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies¹.
- ☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Authored by:

If the work was compiled in a group, the names of all authors are required.

Last name(s):

First name(s):

With my signature I confirm the following:

- I have adhered to the rules set out in the [Citation Guidelines](#).
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date

Signature(s)

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

¹ For further information please consult the ETH Zurich websites, e.g. <https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html> and <https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html> (subject to change).